Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor:	M.C Rene Adrian Davila Perez
Asignatura:	Programación orientada a objetos
Grupo:	7
No de Práctica(s):	11
Integrante(s):	323036285 323279866 323209432 323161891 323063609
No. De Lista o brigada	3
Semestre:	2027-1
Fecha de entrega:	13/11/2026
Obervaciones:	

CALIFICACIÓN: _____

Índice

1. Introducción **2**

2. Marco Teórico **3**

2.1. Manejo de Archivos y Flujos de Bytes con `FileInputStream` . . 3

2.2. Procesamiento de Cadenas con `StringTokenizer` 3

2.3. Cuadros de Texto en Interfaces Gráficas: Swing y JavaFX . . . 4

3. Desarrollo **5**

3.1. Ejercicio 1 5

3.2. Ejercicio 2 5

3.2.1. Lectura del archivo en `ArchivoTexto` y gestión de excepciones 6

3.2.2. Construcción de la Interfaz Gráfica en la clase `Main` . . 7

4. Resultados **9**

5. Conclusiones **10**

1. Introducción

Al comenzar a trabajar con archivos `.txt` en programación, se presenta la dificultad de leer su contenido por flujos de entrada, controlar posibles fallas y contar sus palabras, además del reto de separar la lógica de lectura respecto a la ventana gráfica para evitar un código desordenado en una sola clase, lo cual puede frustrar e incluso confundir al estudiante al ser los archivos complicados a la hora de su manipulación.

Por otra parte, al iniciar en el desarrollo de interfaces gráficas en Java, el estudiantado suele consultar modelos de lenguaje (LLM) para conocer cómo construir cuadros de texto dentro de bibliotecas gráficas como Swing y JavaFX. Sin embargo, las respuestas obtenidas pueden llegar a ser confusas, lo que puede frustrar a los estudiantes.

Resolver ambos problemas nos ayudará a desarrollar criterio técnico para comparar y aprovechar propuestas de una IA sobre cuadros de texto en Swing y JavaFX, así como a integrar el manejo de flujos de bytes y la división de cadenas en tokens vistos en clase para construir una aplicación sencilla, limpia y funcional en Java.

Objetivos

- Consultar con un LLM el uso de cuadros de texto en interfaces gráficas de Java con Swing y JavaFX, para comparar sus componentes, discernir lo útil y seleccionar la alternativa más conveniente.
- Realizar un programa donde se lea un archivo con extensión `.txt` y se cuenten sus palabras, para comprender el trabajo con archivos en Java.
- Desplegar el contenido del archivo y el total de palabras dentro de una ventana gráfica construida con Swing, para aprender a usar las interfaces gráficas en Java.

2. Marco Teórico

2.1. Manejo de Archivos y Flujos de Bytes con `FileInputStream`

En Java, el intercambio de datos entre un programa y una fuente externa, como un archivo de texto plano, se realiza mediante *streams* o flujos de datos ubicados en el paquete `java.io` [1]. Estos se dividen en flujos de bytes, derivados de `InputStream` y `OutputStream`, y flujos de caracteres, derivados de `Reader` y `Writer`.

Dentro de la jerarquía de flujos de entrada de bytes se encuentra la clase `FileInputStream`, la cual abre una conexión hacia un archivo para leer su contenido como secuencia de bytes [1, 2]. Para guardar los datos en un arreglo de bytes, dispone del método `read(byte[] b, int off, int len)`, que almacena la lectura en el arreglo desde la posición inicial indicada y devuelve la cantidad real de bytes leídos, o -1 cuando se alcanza el fin del flujo (por ejemplo, si el archivo está vacío). Al terminar, se debe cerrar la conexión con `close()` para liberar el recurso, además de capturar o propagar la excepción comprobada `IOException` que pueden lanzar estas operaciones [2].

2.2. Procesamiento de Cadenas con `StringTokenizer`

Una vez convertidos los bytes leídos en una cadena de tipo `String`, es posible separarla por palabras con la clase `StringTokenizer` del paquete `java.util` [1, 3]. Al recibir únicamente la cadena de texto, utiliza por defecto como delimitadores los espacios en blanco, tabulaciones, saltos de línea, retornos de carro y avances de página (`\f`).

Para conocer la cantidad total de palabras identificadas dentro de la cadena, esta clase ofrece el método `countTokens()`, el cual calcula el número de tokens disponibles sin necesidad de extraerlos uno por uno en un ciclo [3].

2.3. Cuadros de Texto en Interfaces Gráficas: Swing y JavaFX

Para desplegar texto de múltiples líneas dentro de una interfaz gráfica de usuario (GUI) en Java existen principalmente dos bibliotecas orientadas a escritorio [4, 5]:

- **Java Swing (paquete `javax.swing`):** Viene incluida de forma nativa en el JDK estándar sin requerir dependencias externas. Utiliza `JFrame` para la ventana principal, `JTextArea` como cuadro de texto multilínea, `JScrollPane` para incorporar barras de desplazamiento cuando el texto supera el espacio visible y `JLabel` para mostrar etiquetas breves [4].
- **JavaFX (control `TextArea` del paquete `javafx.scene.control`):** Es una plataforma gráfica separada que requiere agregar su SDK o módulos externos en distribuciones OpenJDK estándar. Su clase `TextArea` permite texto multilínea con desplazamiento propio dentro de una escena `Scene` (`javafx.scene`) y una ventana `Stage` (`javafx.stage`) [5].

3. Desarrollo

3.1. Ejercicio 1

Para el primer ejercicio se le preguntó a la IA acerca de los cuadros de texto en GUI, se usó el siguiente *prompt*:

> "Explica cómo crear un cuadro de texto multilínea en una interfaz gráfica (GUI) en Java con Swing y con JavaFX, indicando únicamente las clases y métodos esenciales para mostrar texto que no pueda ser editado por el usuario."

A lo que respondió:

> En Java Swing se utiliza la clase `JFrame` como ventana principal y `JTextArea` para crear el cuadro de texto multilínea. Para impedir que el usuario modifique el contenido se llama a `setEditable(false)`, y para evitar que las líneas largas se corten se activan `setLineWrap(true)` y `setWrapStyleWord(true)`. Como `JTextArea` no cuenta con desplazamiento propio, se introduce dentro de un `JScrollPane` antes de añadirlo a la ventana. En JavaFX se utiliza una ventana `Stage` con una escena `Scene` y un contenedor como `VBox` que aloja un control `TextArea`; la edición se desactiva igualmente con `setEditable(false)`, el ajuste automático de renglones se habilita con `setWrapText(true)` y las barras de desplazamiento se gestionan de forma interna en el propio control.

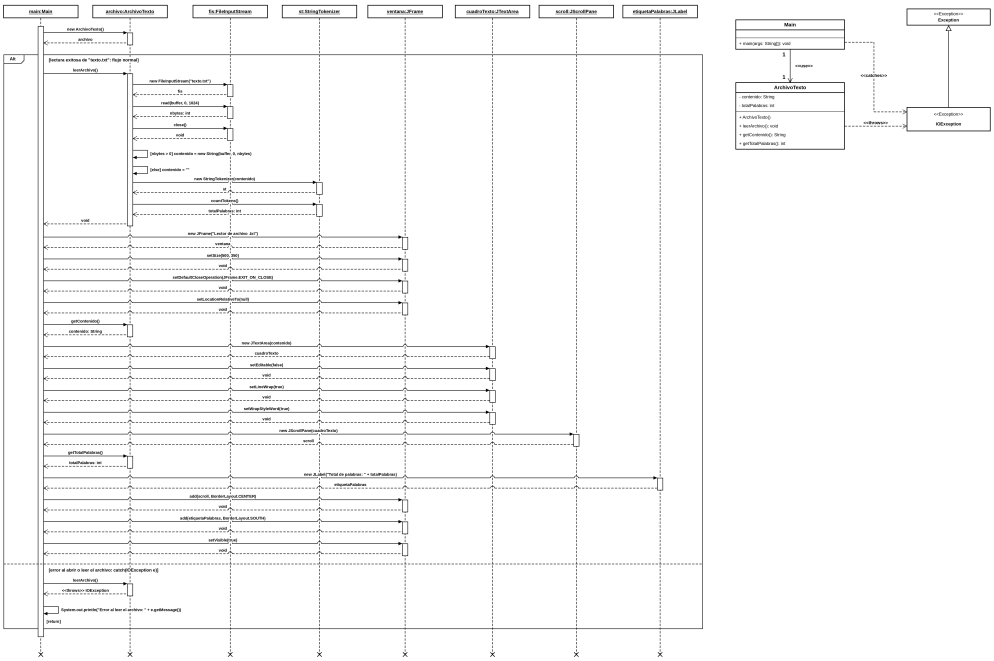
A partir de esta respuesta se compararon ambas alternativas y se optó por emplear Swing en el segundo ejercicio por su disponibilidad nativa en el JDK y su menor configuración inicial.

3.2. Ejercicio 2

Para poner en práctica la selección realizada en el primer ejercicio, el desarrollo de la aplicación se divide en dos fases: primero el procesamiento y

preparación de los datos con la lógica del negocio, y posteriormente la construcción de la ventana gráfica en la clase `Main`.

Para realizar el programa se siguió el siguiente UML.



3.2.1. Lectura del archivo en `ArchivoTexto` y gestión de excepciones

Para manejar la lectura del archivo se creó la clase pública `ArchivoTexto`, con dos atributos privados: `contenido` (`String`) para guardar el texto leído y `totalPalabras` (`int`) para el conteo de palabras. Su constructor público sin parámetros `ArchivoTexto()` inicializa `contenido` con una cadena vacía (`""`) y `totalPalabras` en `0`.

Después se programó el método público `leerArchivo()`, de tipo `void`, el cual declara `throws IOException` para enviar cualquier fallo de lectura hacia la clase principal. Dentro de este método se crea un arreglo `byte[] buffer` de tamaño `1024`, se abre el archivo `"texto.txt"` con `FileInputStream`, se leen los bytes con `fis.read(buffer, 0, 1024)` guardando la cantidad leída en la

variable `nbytes`, y se cierra el flujo con `fis.close()`.

Enseguida, con un `if-else`, si `nbytes >0` se convierte el arreglo a cadena con `new String(buffer, 0, nbytes)` y se guarda en `this.contenido`; de lo contrario, se asigna `""`. Luego se instancia `StringTokenizer` sobre `this.contenido` y se guarda el resultado de `countTokens()` en `this.totalPalabras`. Por último, se agregaron únicamente los getters públicos `getContenido()` y `getTotalPalabras()`, sin setters.

Antes de mostrar cualquier elemento visual en `Main`, el programa inicializa la lectura de la información utilizando un objeto de `ArchivoTexto`. Todo este proceso se encuentra protegido por un bloque `try-catch` preparado para interceptar posibles excepciones de tipo `IOException`: si llega a ocurrir una falla de lectura, el programa imprime un mensaje descriptivo en la consola mediante `System.out.println` concatenando `"Error al leer el archivo: "` con `e.getMessage()`, finalizando la ejecución de inmediato mediante un `return` para evitar crear una ventana vacía o errónea. Una vez completada la lectura con éxito, los datos del archivo quedan listos y almacenados para ser presentados.

3.2.2. Construcción de la Interfaz Gráfica en la clase `Main`

Al llegar al final del proceso de lógica, la clase principal (`Main`) se encarga de albergar el método `public static void main(String[] args)` para construir la ventana gráfica aplicando los componentes obtenidos de la consulta con la IA en el Ejercicio 1 (`JFrame`, `JTextArea` y `JScrollPane`), complementados con `JLabel` y `BorderLayout`.

Siguiendo la elección de Swing, se aprovechó su disponibilidad nativa en el JDK sin configurar dependencias externas, ejecutando la creación de la interfaz directamente en el flujo principal. Para ello, se instanció un objeto `JFrame` configurado con el título `"Lector de archivo .txt"`, dimensiones definidas mediante `setSize(500, 350)`, operación de cierre `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` y centrado en pantalla con `setLocationRelativeTo(null)`.

Posteriormente se creó el cuadro de texto `JTextArea` utilizando el `String` devuelto por `archivo.getContenido()`, configurándolo con `setEditable(false)`,

`setLineWrap(true)` y `setWrapStyleWord(true)` tal como sugirió la IA, y colocándolo dentro de un objeto `JScrollPane` para otorgarle barras de desplazamiento. A la par, se creó una etiqueta `JLabel` concatenando el texto `"Total de palabras: "` con el valor numérico obtenido de `archivo.getTotalPalabras()`.

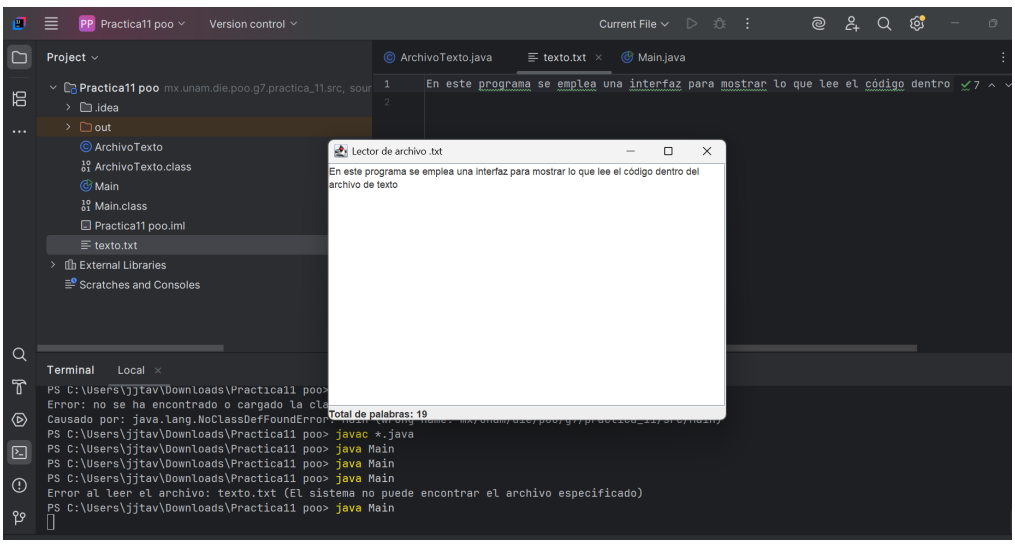
Finalmente, se aprovechó el gestor de diseño predeterminado del `JFrame` (`BorderLayout`) para organizar los componentes sin necesidad de instanciarlos de forma compleja: se pasó la constante `BorderLayout.CENTER` para alojar el `JScrollPane` con el cuadro de texto al centro y `BorderLayout.SOUTH` para situar la etiqueta informativa en la parte inferior. Con todo acomodado, se hizo visible la ventana al usuario mediante `setVisible(true)`.

4. Resultados

Tras concluir el desarrollo del código, realizamos las pruebas de validación necesarias para asegurar los resultados. Para iniciar se hizo una prueba sin el archivo de texto:

```
PS C:\Users\jjtav\Downloads\Practica11 poo> java Main
Error al leer el archivo: texto.txt (El sistema no puede encontrar el archivo especificado)
PS C:\Users\jjtav\Downloads\Practica11 poo>
```

Posteriormente se preparó el archivo de texto plano `texto.txt`.



El cuadro de texto desplegó el contenido con ajuste automático por palabras y edición bloqueada, mientras que la etiqueta inferior mostró `"Total de palabras: 19"`, lo cual comprobó visualmente tanto la lectura completa del texto con `FileInputStream` como el conteo exacto de tokens con `StringTokenizer`.

5. Conclusiones

Se cumplieron los objetivos de la práctica. En primer lugar, se cumplió el primer objetivo ya que se consultó a un modelo de lenguaje (LLM) sobre la creación de cuadros de texto en interfaces gráficas con Swing y JavaFX, lo que permitió comparar ambas alternativas y seleccionar Swing por su disponibilidad nativa en el JDK. En segundo lugar, se alcanzó el segundo objetivo al implementar la lectura del archivo `texto.txt` mediante flujos de bytes con `FileInputStream` y el conteo de palabras con `StringTokenizer`, logrando controlar posibles fallas con `IOException`. Finalmente, se cumplió el tercer objetivo al desplegar el contenido y el total de palabras dentro de una ventana gráfica construida con `JFrame`, `JTextArea`, `JScrollPane` y `JLabel`, comprobando que separar la lectura del archivo de la interfaz facilita la organización del programa.

Referencias

[1] M. A. Nakayama C. y J. A. Solano Gálvez, "Guía práctica de estudio 11: Manejo de archivos," en *Manual de prácticas del Laboratorio de Programación Orientada a Objetos*, Código MADO-22, Versión 01, México: Facultad de Ingeniería UNAM, ene. 2017, pp. 165–176.

[2] Oracle Corporation, "Class FileInputStream," *Java Platform, Standard Edition 8 API Specification*, 2014. [En línea]. Disponible en: `https://docs.oracle.com/javase/8/docs/api/java/io/FileInputStream.html`. [Accedido: 11-nov-2026].

[3] Oracle Corporation, "Class StringTokenizer," *Java Platform, Standard Edition 8 API Specification*, 2014. [En línea]. Disponible en: `https://docs.oracle.com/javase/8/docs/api/java/util/StringTokenizer.html`. [Accedido: 11-nov-2026].

[4] Oracle Corporation, "How to Use Text Areas," *The Java Tutorials*, s.f. [En línea]. Disponible en: `https://docs.oracle.com/javase/tutorial/uiswing/components/textarea.html`. [Accedido: 11-nov-2026].

[5] OpenJFX, "Class TextArea," *JavaFX 21 API Specification*, 2023. [En línea]. Disponible en: `https://openjfx.io/javadoc/21/javafx.controls/javafx/scene/control/TextArea.html`. [Accedido: 11-nov-2026].